

1. ID3 / DECISION TREE

$$H(S)=-\sum p(c)\log_2 p(c)$$
$$AE(S,A)=\sum_v |S_v|/|S| \cdot H(S_v)$$
$$IG(S,A)=H(S)-AE(S,A)$$

$$SplitInfo=-\sum_v |S_v|/|S| \cdot \log_2(|S_v|/|S|)$$
$$GainRatio=IG/SplitInfo$$

Dựng cây

- H tập hiện tại -> AE/IG (hoặc GR) từng thuộc tính -> chọn lớn nhất -> chia subset -> lặp.
- Dừng: subset rỗng -> majority parent; cùng class -> leaf; hết attribute -> majority hiện tại.
- Mỗi root-to-leaf = một IF-THEN rule; phân loại mẫu bằng cách đi đúng nhánh.

**Bấy:** trọng số  $|S_v|/|S|$ ;  $0\log 0=0$ ; IG bias thuộc tính nhiều giá trị; dừng làm tròn sớm.

**Ghi bài:** bảng [attribute | entropy từng nhánh | AE | IG/GR], rồi vẽ cây.

2. NAIVE BAYES

$$P(C|X)=P(X|C)P(C)/P(X)$$
$$score(C)=P(C)\prod_i P(x_i|C) \rightarrow \text{chọn score max.}$$

$$Laplace: P(x_i=v|C)=(count(v,C)+1)/(count(C)+k),$$
$$k=\text{\# giá trị có thể của thuộc tính.}$$

- Prior  $P(C)=count(C)/N$  -> conditional -> nhân score. So class thì bỏ  $P(X)$ .
- Bấy: không đảo  $P(x|C)$ ; không bỏ prior; xác suất 0 -> Laplace; mẫu số là  $count(C)+k$ .

**Ưu tiên để cũ: ID3 3/3; Logic 3/3; sau đó Minimax, NB, CSP.**

3. PROPOSITIONAL LOGIC

$$P \rightarrow Q \equiv \neg P \vee Q$$
$$P \leftrightarrow Q \equiv (\neg P \vee Q) \wedge (\neg Q \vee P)$$
$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$
$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$
$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$

Chứng minh KB  $\models \alpha$  bằng resolution

- Lập KB  $\wedge \neg \alpha$  -> khử  $\leftrightarrow$ ,  $\rightarrow$  -> đẩy  $\neg$  -> phân phối  $\vee$  qua  $\wedge$  -> clauses.
- Resolve:  $(A \vee P), (B \vee \neg P) \Rightarrow (A \vee B)$ . Ra  $\square$  thì entailment đúng.
- Ghi số clause và hai clause cha. Không resolve literal cùng dấu.

Forward chaining

- Facts -> rule có mọi premise đã biết -> thêm conclusion -> lặp đến query/fixed point.

Backward chaining

- Query -> tìm rule kết luận ra query -> biến premises thành subgoals -> facts; tránh vòng lặp.
- Bấy: refutation thêm  $\neg$  query;  $\square$  là mâu thuẫn; CNF = AND của clauses, clause = OR của literals.**

4. FOL - DỊCH NHANH

- mọi A là B:  $\forall x(A(x) \rightarrow B(x))$
- có A là B:  $\exists x(A(x) \wedge B(x))$
- không A nào là B:  $\forall x(A(x) \rightarrow \neg B(x))$
- chỉ A mới B:  $\forall x(B(x) \rightarrow A(x))$  - coi chừng chiều.

5. FOL - CNF / UNIFICATION

FOL -> CNF

- Khử  $\leftrightarrow$ ; khử  $\rightarrow$ ; đẩy  $\neg$  qua lượng từ; standardize apart; Skolemize  $\exists$ ; bỏ  $\forall$ ; phân phối  $\vee$  qua  $\wedge$ ; tách clause.

$$\neg \forall x P \equiv \exists x \neg P; \neg \exists x P \equiv \forall x \neg P$$

**Skolem:**  $\exists x P(x) \rightarrow P(c)$ .  
 $\forall y \exists x P(x,y) \rightarrow P(f(y),y)$ .  
**Mỗi  $\exists$  độc lập dùng symbol mới; function nhận các  $\forall$  bao ngoài.**

Unification / MGU

- Tìm  $\theta$  để  $E1\theta=E2\theta$ . Constant khác nhau -> fail. Function khác name/arity -> fail. Occurs check: x không unify  $f(x)$ .
- Áp dụng substitution mới vào toàn bộ biểu thức/substitution đang có.

**GMP:**  $p1', ..., pn', (p1 \wedge ... \wedge pn \rightarrow q)$ , với  $p_i' = p_i \theta \Rightarrow q \theta$ .

FOL resolution

- Standardize apart -> chọn literal đối ngẫu -> unify bằng MGU  $\theta$  -> bỏ cặp -> áp  $\theta$  lên resolvent -> lặp đến  $\square$ .
- Trình bày:  $C1, C2, \theta = \{...\} \Rightarrow C3$ .

**Bấy: thiếu  $\theta$ ; dùng chung Skolem constant; quên biến  $\forall$  trong Skolem function; resolve trước khi standardize.**

6. MINIMAX / ALPHA-BETA

**MAX** lấy max child; **MIN** lấy min child.  
**Time**  $O(b^m)$ , **space**  $O(bm)$ .

**MAX:**  $v=-\infty$ ;  $v=\max(v, child)$ ; nếu  $v \geq \beta$  **prune**;  
 $\alpha=\max(\alpha, v)$ .  
**MIN:**  $v=+\infty$ ;  $v=\min(v, child)$ ; nếu  $v \leq \alpha$  **prune**;  
 $\beta=\min(\beta, v)$ .

- Duyệt DFS đúng thứ tự để. Truyền  $(\alpha, \beta)$  xuống. Cập nhật v sau mỗi child, kiểm tra cắt ngay.
- Ghi  $\alpha, \beta$  cạnh node; chỉ gạch các child chưa duyệt. Root chọn action có minimax value tốt nhất.
- Alpha-beta không đổi kết quả. Move ordering càng tốt càng cắt nhiều; best case  $\sim O(b^{(m/2)})$ .

**Cutoff: đến depth limit thì dùng EVAL thay UTILITY; EVAL phải tương quan kết quả thật.**

7. CSP

**CSP = variables X + domains D + constraints C.**

- Backtrack: complete? -> choose variable -> order values -> consistent? assign+recurse -> undo.
- MRV: domain nhỏ nhất. Degree: tie-break, nhiều neighbor chưa gán nhất. LCV: loại ít lựa chọn neighbor nhất.

**Forward checking:** sau  $X=v$ , xóa giá trị conflict khỏi domain neighbors; domain rỗng -> backtrack.

**AC-3: queue all arcs. REVISE( $X_i, X_j$ ):** xóa  $x \in D_i$  nếu không có  $y \in D_j$  thỏa constraint. Di rỗng -> fail; nếu đổi thì thêm  $(X_k, X_i)$ ,  $k \neq j$ .

- Bấy: REVISE có hướng; sau mỗi revise phải ghi domain mới; FC yếu hơn AC-3.

8. PERCEPTRON / BACKPROP

**Perceptron**  
 $net = \sum w_i x_i - \theta$ ;  $y = \text{step}(net)$   
 $e = y_d - y$ ;  $\Delta w_i = \eta x_i e$ ;  $w_i \leftarrow w_i + \Delta w_i$

- Bias có thể là  $x_0=1$ . Một perceptron không giải XOR.

**Sigmoid**  
 $\sigma(z) = 1/(1+e^{-z})$   
 $\sigma'(z) = \sigma(z)(1-\sigma(z))$

**Forward**  
 $net_j = \sum_i w_{ij} x_i$ ;  $y_j = \sigma(net_j)$   
 $E = 1/2 \sum_k (y_d - y_k)^2$

**Backprop**  
 $\delta_k = y_k(1-y_k)(y_d - y_k)$   
 $\delta_j = y_j(1-y_j) \sum_k w_{jk} \delta_k$   
 $\Delta w_{ij} = \eta \delta_j x_i$ ;  $w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$

- Thứ tự: forward -> output delta -> hidden delta bằng weight cũ -> update toàn bộ.
- Bấy: quên bias; đảo dấu target-output; update sớm; sai chiều index weight; làm tròn sớm.

## 9. SEARCH - CÔNG THỨC

**UCS:  $f=g$ . Greedy:  $f=h$ . A\*:  $f=g+h$ .**

- Admissible:  $0 \leq h(n) \leq h^*(n)$ . Consistent:  $h(n) \leq c(n, n') + h(n')$ .
- Consistent  $\Rightarrow$  admissible ( $h(\text{goal})=0$ ). A\* graph search optimal nếu h consistent.

## 10. SEARCH - SO SÁNH

**BFS: FIFO; complete; optimal nếu step cost đều; space lớn.**

**DFS: LIFO; không optimal; có thể không complete.**

**UCS: min q; optimal với  $\text{cost} \geq \varepsilon$ ; goal test khi POP.**

**Greedy:** min h; nhanh nhưng không đảm bảo optimal.

**A\*:**  $\min g+h$ ; graph search optimal với  $h$  consistent.

- UCS/A\*: dùng khi goal được lấy khỏi priority queue, không phải lúc sinh.
- Có path rẻ hơn tới node: update/reinsert frontier. Tie-break theo đề.
- Ghi expanded order riêng returned path; ghi frontier nếu đề yêu cầu.

## 11. PEAS / AGENT

**PEAS = Performance, Environment, Actuators, Sensors.**

- Simple reflex: current percept. Model-based: internal state. Goal-based: goal/search. Utility-based: expected utility. Learning: experience/feedback.

## 12. LOCAL SEARCH

- Hill climbing: best neighbor; kẹt local max/ridge/plateau.
- Simulated annealing: đôi lúc nhận bước xấu; temperature giảm dần.
- Local beam: giữ k states. GA: selection -> crossover -> mutation; cần representation/fitness tốt.

## TRÌNH BÀY ĂN ĐIỂM

- Công thức trước, thay số sau; giữ  $\geq 4$  số lẻ trung gian.
- ID3: bảng H/AE/IG. Logic: clause cha +  $\theta$ . Alpha-beta:  $\alpha/\beta$ . CSP: domains. NN: net->y-> $\delta$ ->update.
- Nếu ket, vẫn ghi quy trình và đại lượng trung gian.

**Slide: L10 ID3/NB; L7 PL; L8 FOL; L5 game; L6 CSP; L11 NN; L3 search; L2 agents; L4 local search.**

## GHI CHÚ SAU KHI LÀM ĐỀ

Dùng cột này để ghi đúng lỗi mình vừa mắc; càng cụ thể càng dễ tránh lặp lại.

- ☐ Công thức/quy tắc mình quên
- ☐ Bước tính hoặc phép biến đổi mình sai
- ☐ Quy ước duyệt/tie-break của đề
- ☐ Lỗi trình bày làm mất điểm

[illegible]